Category C — Coding Assignment

Problem 4: Exam Score Band Counter (Completed Solution)

Author: University Student | **Course:** Data Structures and Algorithms

Task Description

A university stores the marks of all students in an exam as an array sorted in ascending order; marks can repeat. Faculty members frequently ask how many students scored within a band [low, high], inclusive of both ends.

Implement a function countInBand(scores, low, high) that returns the number of scores *s* such that low <= s <= high. The array is large and queried very often, so your solution must not scan the scores one by one.

Constraints & Examples

Constraints:
• $1 \le$ length of scores $\le 10^6$
• scores is sorted in ascending order
• $0 \le$ low $\le$ high ≤ 100

Sample 1:
scores = [35, 42, 42, 50, 58, 58, 58, 63, 71, 88], low = 42, high = 58
Expected Output: 6

Sample 2:
scores = [35, 42, 42, 50, 58, 58, 58, 63, 71, 88], low = 90, high = 100
Expected Output: 0

Optimal Python Solution (Binary Search)

```
import bisect

def countInBand(scores: list[int], low: int, high: int) -> int:
    """
    Returns the number of scores s such that low <= s <= high
    in a sorted list of scores using binary search (bisect).
    """
    # Find the first index where element is >= low
    left_idx = bisect.bisect_left(scores, low)

    # Find the first index where element is > high
    right_idx = bisect.bisect_right(scores, high)

    # The number of elements in the range [low, high]
    return right_idx - left_idx
```

Complexity Analysis & Edge Case Explanation

1. Time and Space Complexity:

• **Linear Scan:** Time complexity is **O(N)** where N is the length of the scores array (up to 10^6). Since faculty members query the array very often, scanning element by element for every query is highly inefficient. Space complexity is **O(1)**.

• **Final Solution (Binary Search):** Time complexity is **O(log N)** per query because it performs two binary searches using bisect_left and bisect_right. Space complexity is **O(1)** auxiliary space.

2. How Duplicates at the Edges Affect the Search:

Because marks can repeat (e.g., multiple students scoring 42 or 58), standard binary search which stops at the first matching element encountered is insufficient. To accurately count all elements in [low, high]:

• We must use **lower bound (bisect_left)** to locate the exact starting position of the first occurrence of an element ≥ low.

• We must use **upper bound (bisect_right)** to locate the position immediately after the last occurrence of an element ≤ high.

• Subtracting left_idx from right_idx correctly accounts for all duplicate entries residing at or near the boundary edges.